Частина 2

Перед виконання 4, 5 та 6 лабораторних робіт слід налаштувати середовище слідуючи інструкціям додатку А для коректної компіляції та використання в коді CUDA Runtime. Додатково для виконання 5 і 6 роботи слід під’єднати відповідні бібліотеки(CUBLAS та CUFFT).

ЛАБОРАТОРНА РОБОТА №6

Назва роботи: Знайомство з технологією CUDA

Мета роботи: Засвоїти основи роботи з CUDA.

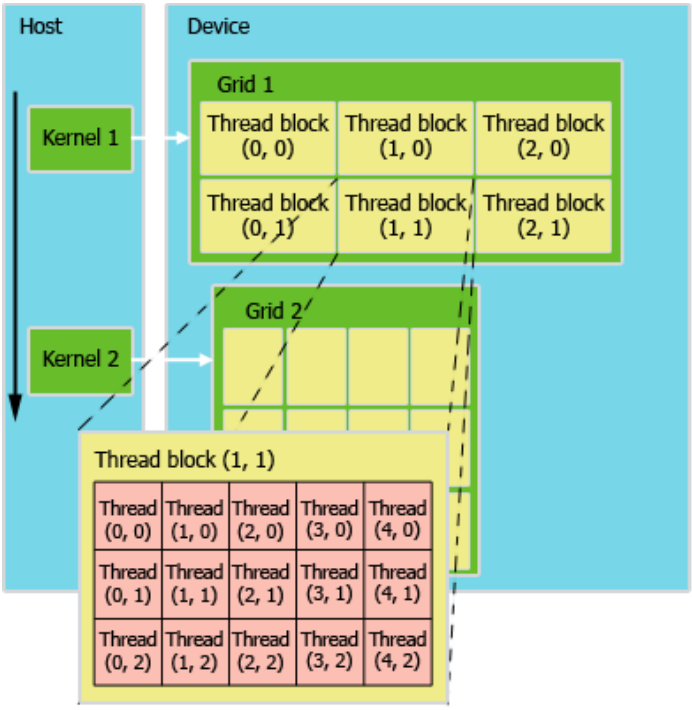
1. Загальні відомості

Всі інструкції в відеокарті виконуються за принципом SIMD, коли одна інструкція застосовується до всіх потоків(в CUDA це група з 32 потоків - мінімальний обсяг даних, що обробляють мультипроцесори). Цей спосіб виконання назвали SIMT (один потік команд декілька потоків даних).

При виконанні програми, центральний процесор виконує свої порції коду, а GPU виконує CUDA код з найбільш важкими паралельними обчисленнями. Ця частина, призначена для GPU, називається ядром. У ядрі визначаються операції, які будуть виконані над даними.

Таким чином, CUDA використовує паралельну модель обчислень, коли кожен з SIMD процесорів виконує ту ж інструкцію над різними елементами даних паралельно. GPU є обчислювальним пристроєм(device) для центрального процесора(host), що володіє власною пам'яттю і обробляює паралельно велику кількість потоків. Ядром називається функція для GPU, що виконується потоками. Відеочіп відрізняється від CPU тим, що може обробляти одночасно сотні тисяч потоків, що є звичним для графіки, яка добре розпаралелюється.

Модель програмування в CUDA передбачає групування потоків. Потоки об'єднуються в блоки потоків(block) — одномірні або двовимірні сітки потоків, що взаємодіють між собою за допомогою спільної пам'яті і точок синхронізації. Програма (ядро) виконується над сіткою(grid) блоків потоків, див. малюнок нижче. Одночасно виконується одна сітка. Кожен блок може бути одно-, дво- або тривимірним за формою, і може складатися з багатьох потоків на потоковому апаратному забезпеченні.



2. Приклад програми

Як приклад розглянемо задачу множення квадратної матриці на вектор.
Вихідні дані:
A[n][n] – матриця розмірності n x n;
b[n] – вектор, що складається з n елементів.
Результат:
c[n] – вектор із n елементів.

```
// Послідовний алгоритм множення матриці на вектор
for (i=0; i<n; i++)
{
        c[i]=0;
        for (j=0; j<m; j++)
        {
            c[i]=A[i][j]*b[j];
        }
}
```

Тепер розглянемо рішення цієї задачі на відеокарті. Наступний код ілюструє приклад виклику функції CUDA:

```
// ініціалізація CUDA
if(!InitCUDA())   { return 0; }

int Size = 1000;
// звичайні масиви в оперативній пам'яті
float *h_a,*h_b,*h_c;
h_a = new float[Size*Size];
h_b = new float[Size];
h_c = new float[Size];

for (int i=0;i<Size;i++)  // ініціалізація масивів a і b
{
```

```
  for (int k=0;k<Size;k++)
     {
       h_a[i*Size+k]=1;
     }
  h_b[i]=2;
}

// вказівники на масиви в відеопам'яті
float *d_a,*d_b,*d_c;

// виділення відеопам'яті
cudaMalloc((void **)&d_a, sizeof(float)*Size*Size);
cudaMalloc((void **)&d_b, sizeof(float)*Size);
cudaMalloc((void **)&d_c, sizeof(float)*Size);

// копіювання з оперативної пам'яті в відеопам'ять
CUDA_SAFE_CALL(cudaMemcpy(d_a, h_a, sizeof(float)*Size*Size,
                              cudaMemcpyHostToDevice) );
CUDA_SAFE_CALL(cudaMemcpy(d_b, h_b, sizeof(float)*Size,
                              cudaMemcpyHostToDevice) );

// встановлення кількості блоків
dim3 grid((Size+255)/256, 1, 1);
// встановлення кількості потоків в блоці
dim3 threads(256, 1, 1);

// виклик функції
MatrVectMul<<< grid, threads >>> (d_c, d_a, d_b,Size);

// копіювання з відеопам'яті в оперативну пам'ять
CUDA_SAFE_CALL(cudaMemcpy(h_c, d_c, sizeof(float)*Size,
                              cudaMemcpyDeviceToHost) );

// звільнення пам'яті
CUDA_SAFE_CALL(cudaFree(d_a));
CUDA_SAFE_CALL(cudaFree(d_b));
CUDA_SAFE_CALL(cudaFree(d_c));
```

В принципі структуру будь-якої програми з використанням CUDA можна представити аналогічно розглянутому вище прикладу. Таким чином, можна запропонувати наступну послідовність дій:

1) ініціалізація CUDA
2) виділення відеопам'яті для зберігання даних програми
3) копіювання необхідних для роботи функції даних з оперативної пам'яті в відеопам'ять
4) виклик функції CUDA
5) копіювання результуючих даних з відеопам'яті в оперативну
6) звільнення відеопам'яті

Приклад функції, що виконується на відеокарті

```
extern "C" __global__ void MatrVectMul(float *d_c, float *d_a,
float *d_b, int Size)
```

```
{

        int i = blockIdx.x*blockDim.x+threadIdx.x;
        int k;
        d_c[i]=0;
        for (k=0;k<Size;k++)
          {
             d_c[i]+=d_a[i*Size+k]*d_b[k];
          }
}
```

Тут: `threadIdx.x` – ідентифікатор потоку в блоці по координаті x, `blockIdx.x` – ідентифікатор блоку в гріді по координаті x, `blockDim.x` – кількість потоків в одному блоці.

Таким чином виходить унікальний ідентифікатор потоку, який можна використовувати в програмі, працюючи з усіма потоками як з одновимірним масивом.

Важливо пам'ятати, що функція, призначена для виконання на відеокарті, не повинна звертатися до оперативної пам'яті. Подібне звернення призведе до помилки. Якщо необхідно працювати з будь-яким об'єктом в оперативній пам'яті, попередньо його треба скопіювати в відеопам'ять, і звертатися з функції CUDA до цієї копії.

Для вимірювання часу виконання окремих частин програми і аналізу швидкості виконання тієї чи іншої ділянки коду зручно використовувати вбудовані в CUDA функції виміру часу.

```
unsigned int timer;

// створення таймеру
cutCreateTimer(&timer);

// запуск таймеру
cutStartTimer(timer);

// код …

// зупинка таймеру
cutStopTimer( timer);

// отримання значення часу
printf("time: %f (ms)\n", cutGetTimerValue(timer));

// видалення таймеру
cutDeleteTimer( timer);
```

3. Приклад програми з використанням спільної пам'яті

Приклад CUDA-ядра (matmulshm), що виконує множення матриць*(у функцію передаються вказівники на перший елемент матриць)*:

```
__global__ void matmulShM(float* a, float* b, int n, float* c)
{
   int bx = blockIdx.x;
   int by = blockIdx.y;
   int tx = threadIdx.x;
   int ty = threadIdx.y;

   // Index of the first sub-matrix of A processed by the block
   int aBegin = n * BLOCK_SIZE * by;
   int aEnd = aBegin + n - 1;

   // Step size used to iterate through the sub-matrices of A
   int aStep = BLOCK_SIZE;

   // Index of the first sub-matrix of B processed by the block
   int bBegin = BLOCK_SIZE * bx;

   // Step size used to iterate through the sub-matrices of B
   int bStep = BLOCK_SIZE * n;

   // Computed subelement
   float sum = 0.0f;
   for (int ia = aBegin, ib = bBegin; ia <= aEnd; ia += aStep, ib += bStep)
   {
      // Shared memory for the sub-matrix of A
      __shared__ float as [BLOCK_SIZE][BLOCK_SIZE];

      // Shared memory for the sub-matrix of B
      __shared__ float bs [BLOCK_SIZE][BLOCK_SIZE];

      // Load the matrices from global memory to shared memory
      as [ty][tx] = a [ia + n * ty + tx];
      bs [ty][tx] = b [ib + n * ty + tx];

      // Synchronize to make sure the matrices are loaded
      __syncthreads();

      // Multiply the two matrices together;
      for (int k = 0; k < BLOCK_SIZE; k++)
         sum += as [ty][k] * bs [k][tx];

      // Synchronize to make sure that the preceding
      // computation is done before loading two new
      // sub-matrices of A and B in the next iteration
      __syncthreads();
   }

   // Write the block sub-matrix to global memory
```

```
   // each thread writes one element
   int ic = n * BLOCK_SIZE * by + BLOCK_SIZE * bx;

   c [ic + n * ty + tx] = sum;
}
```

4. Варіанти завдань

У всіх завданнях потрібно створити матрицю (чи вектор) або кілька матриць (чи векторів) які вказанії в таблиці 1 і виконати для них відповідні обчислення.

Інтерфейс програми — консольний додаток.

Таблиця 1. Варіанти завдань

№	Опис завдання
1	Обчислити матрицю C = ½ * A + B
2	Обчислити матрицю C = (A + B) * 5 + 8
3	Обчислити матрицю C = (B-A) * ½-8
4	Обчислити матрицю C = A * 5 - B * 8 + 3
5	Обчислити добуток матриць C = A * B
6	Обчислити одновимірний масив C = (A-B) * 4 + 5
7	Обчислити одновимірний масив C = (A + B) * 6-10
8	Обчислити одновимірний масив C = (B-A) * ½-8
9	Обчислити одновимірний масив C = A * 3-B + 3
10	Обчислити одновимірний масив C = B + 3 * A-8
11	Обчислити матрицю C = ½ * A + B
12	Обчислити матрицю C = (A + B) * 5 + 8
13	Обчислити матрицю C = (B-A) * ½-8
14	Обчислити матрицю C = A * 5 - B * 8 + 3
15	Обчислити матрицю C = 2 * A * B
16	Обчислити одновимірний масив C = (A-B) * 4 + 5
17	Обчислити одновимірний масив C = (A + B) * 6-10
18	Обчислити одновимірний масив C = (B-A) * ½-8
19	Обчислити одновимірний масив C = A * 3-B + 3
20	Обчислити одновимірний масив C = B + 3 * A-8
21	Обчислити матрицю C = 3 * A * B
Матриці квадратичні і мають одинаковий розмір. *Пропонується матлабівське трактування запису (A + n) як A + матриця розміру того ж що і матриця A з елементами значення яких рівні n (*замість звичного (A + n * E), де A - матриця, E - одинична матриця, n – число*)	

5. Зміст звіту

- Титульний лист;
- Завдання;
- Алгоритм рішення завдання;
- Код програми;
- Екранна форма з результатами роботи програми;
- Висновки.

ЛАБОРАТОРНА РОБОТА №7

Назва роботи: Використання бібліотеки CUBLAS

Мета роботи: Оволодіти навичками роботи з бібліотекою CUBLAS

** до роботи додається документація (**файл: CUBLAS_Library.pdf**) з детальною інформація про бібліотеку*

1. Загальні відомості

Basic Linear Algebra Subprograms (BLAS) (укр. Основні підпрограми з лінійної алгебри) — встановлений де-факто стандарт інтерфейсу бібліотек підпрограм, призначених для виконання основних операцій лінійної алгебри, таких як, наприклад, множення матриць та векторів. Уперше такі підпрограми було опубліковано 1979 року. На їх основі будуються більші пакети, такі як LAPACK. Конкретні втілення інтерфейсу у вигляді бібліотек розробляються як провідними лідерами комп'ютерної технології, наприклад, Intel-ом, так і іншими авторами. Підпрограми лінійної алгебри широко використовуються в застосуваннях, які вимагають інтенсивних обчислень, а тому вимагають особливо уважної розробки й оптимізації. Наприклад, ATLAS — варіант BLAS, який може легко переноситися з однієї платформи на іншу й оптимізуватися для використання на машинах конкретної архітектури.

Є безліч оптимізованих реалізацій на Fortran і C, у тому числі від виробників пристроїв:

- Goto BLAS
- BLAS у складі Intel MKL
- BLAS у складі ACML
- CUBLAS у складі CUDA Toolkit з версії 1.0

∗Бібліотека CUBLAS орієнтований на Фортран. Тому з метою сумісності з існуючими версіями в матрицях послідовно лежать елементи в стовпцях (column-major storage format)
∗Нумерація елементів починається з одиниці.
∗Для C \ C ++ треба використовувати макроси:
∗ # define IDX2F (i, j, ld) ((((j) -1) * (ld)) + ((i) -1))
∗ # define IDX2C (i, j, ld) (((j) * (ld)) + (i))

<u>CUBLAS-іменування функцій:</u> **cublas <T> <func>**
• <T> - тип даних (S, D, C, Z) відповідає дійсним числам з одинарною подвійною точністю і комплексним числам одинарної і подвійної точності
• <func> - 3x-4x буквена назва функції BLAS
• Приклад: cublasDgemm

Послідовність використання бібліотеки CUBLAS

- Ініціалізувати дескриптор (*cublasCreate()*)
- Виділити пам'ять і завантажити дані
- Викликати послідовність функцій CUBLAS
- Копіювати результат в оперативну пам'ять
- Звільнити дескриптор (*cublasDestroy()*)

2. Приклад програми

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <cuda_runtime.h>
#include "cublas_v2.h"
#define M 6
#define N 5
#define IDX2C(i,j,ld) (((j)*(ld))+(i))
static __inline__ void modify (cublasHandle_t handle, float *m, int ldm, int
n, int p, int q, float alpha, float beta){
    cublasSscal (handle, n-p, &alpha, &m[IDX2C(p,q,ldm)], ldm);
    cublasSscal (handle, ldm-p, &beta, &m[IDX2C(p,q,ldm)], 1);
}

int main (void){
    cudaError_t cudaStat;
    cublasStatus_t stat;
    cublasHandle_t handle;
    int i, j;
    float* devPtrA;
    float* a = 0;
    a = (float *)malloc (M * N * sizeof (*a));
    if (!a) {
        printf ("host memory allocation failed");
        return EXIT_FAILURE;
    }
    for (j = 0; j < N; j++) {
        for (i = 0; i < M; i++) {
            a[IDX2C(i,j,M)] = (float)(i * M + j + 1);
        }
    }
    cudaStat = cudaMalloc ((void**)&devPtrA, M*N*sizeof(*a));
    if (cudaStat != cudaSuccess) {
        printf ("device memory allocation failed");
        return EXIT_FAILURE;
    }
    stat = cublasCreate(&handle);
    if (stat != CUBLAS_STATUS_SUCCESS) {
        printf ("CUBLAS initialization failed\n");
        return EXIT_FAILURE;
    }
    stat = cublasSetMatrix (M, N, sizeof(*a), a, M, devPtrA, M);
    if (stat != CUBLAS_STATUS_SUCCESS) {
        printf ("data download failed");
        cudaFree (devPtrA); cublasDestroy(handle);
        return EXIT_FAILURE;
    }
    modify (handle, devPtrA, M, N, 1, 2, 16.0f, 12.0f);
    stat = cublasGetMatrix (M, N, sizeof(*a), devPtrA, M, a, M);
    if (stat != CUBLAS_STATUS_SUCCESS) {
        printf ("data upload failed");
        cudaFree (devPtrA);
        cublasDestroy(handle);
        return EXIT_FAILURE;
    }
    cudaFree (devPtrA);
    cublasDestroy(handle);
    for (j = 0; j < N; j++) {
        for (i = 0; i < M; i++) {
            printf ("%7.0f", a[IDX2C(i,j,M)]);
        }
```

```
        printf ("\n");
    }
    free(a);
    return EXIT_SUCCESS;
}
```

3. Варіанти завдань

Реалізувати з використанням CUBLAS варіанти завдання з четвертої лабораторної роботи.

4. Зміст звіту

- Титульний лист;
- Завдання;
- Алгоритм рішення завдання;
- Код програми;
- Екранна форма з результатами роботи програми;
- Висновки

ЛАБОРАТОРНА РОБОТА №8

Назва роботи: Використання бібліотеки CUFFT

Мета роботи: Оволодіти навичками роботи з бібліотекою CUFFT

** до роботи додається документація (**файл: CUFFT_Library.pdf**) з детальною інформація про бібліотеку*

*** перед виконанням лабораторної роботи студентам рекомендується(хоча це не є необхідною умовою) ознайомитися з бібліотекою fftw*

1. Загальні відомості

Швидке перетворення Фур'є (часто FFT від англ. Fast Fourier Transform) — швидкий алгоритм обчислення дискретного перетворення Фур'є. Якщо для прямого обчислення дискретного перетворення Фур'є з N точок даних потрібно O(N^2) арифметичних операцій, то FFT дозволяє обчислити такий самий результат використовуючи O(N*log(N)) операцій. Алгоритм FFT часто використовується для цифрової обробки сигналів для перетворення дискретних даних з часового у частотний діапазон.

CUFFT реалізація бібліотеки швидкого перетворення Фур'є (ШПФ) FFT (Fast Fourier Transform) для CUDA. ШПФ - це алгоритм ефективного обчислення дискретних перетворень Фур'є для комплексних або дійсних наборів даних. Є одним з найбільш важливих і широко використовуваних чисельних алгоритмів, з додатками, які включають в себе обчислювальну фізику і обробку сигналів. Бібліотека CUFFT надає простий інтерфейс для паралельних обчислень FFT на GPU, що дозволяє користувачам оперувати числами з плаваючою комою, і використовувати паралелізм GPU без необхідності розробки власної реалізації ШПФ орієнтованої на GPU.

FFT бібліотеки зазвичай відрізняються підтримуваними форматами перетворення і типами даних. Наприклад, деякі бібліотеки реалізовують тільки Radix-2 FFT, обмежуючи розмір перетворення другим ступенем, в той час як інші реалізації підтримують перетворення довільних розмірів.

<u>Бібліотека CUFFT підтримує наступні функції:</u>

1. 1D, 2D і 3D перетворення складних і речових даних;
2. пакетне виконання декількох перетворень будь-якої розмірності паралельно;
3. 2D і 3D перетворення в діапазоні [2,16384] в будь-якої розмірності;
4. 1D перетворення до 8 мільйонів елементів;
5. перетворення для дійсних і комплексних даних;
6. подвійна точність перетворення для сумісних пристроїв (GT200 і пізніших версій);
7. підтримка потокового виконання, що дозволяє одночасні обчислення разом зі зміною даних.

Типи даних і типи перетворень

θВикористовуються наступні типи даних:

-cufftReal = Float (скорочено R);

-cufftDoubleReal = Double (скорочено D);

-cufftComplex = Float2 (скорочено C);

-cufftDoubleComplex = Double2 (скорочено Z).

θПідтримуються наступні типи перетворень: CUFFT_R2C, CUFFT_C2R, CUFFT_C2C, CUFFT_D2Z, CUFFT_Z2D, CUFFT_Z2Z.

θНапрям перетворення визначається макросами CUFFT_FORWARD і CUFFT_INVERSE.

Створення і знищення плану

θКажне перетворення описується описується планом (plan).

θПлан має тип cufftHandle.

θДля створення плану використовуються функції:

cufftPlan1d (...), cufftPlan2d (...), cufftPlan3d (...), cufftPlanMany (...).

θcufftResult cufftPlan1d (cufftHandle * plan, int nx, cufftType type, int batch) - створює план одновимірного перетворення типу type, сигналу довжини nx, batch дорівнює кількості перетворень.

θДля знищення плану використовується функція cufftResult cufftDestroy (cufftHandle plan).

Виконання перетворення

θВиконання перетворення відповідного типу із заданим планом виконується функціями cufftExecC2C (...), cufftExecR2C (...), cufftExecC2R (...), cufftExecZ2Z (...), cufftExecD2Z (...), cufftExecZ2D (...).

θcufftResult cufftExecC2C (cufftHandle plan, cufftComplex * idata, cufftComplex * odata, int direction) - виконує перетворення над даними idata із заданим планом plan, результат записується в odata, напрямок визначається direction:

-idata, odata - вказівники на device пам'ять, можуть збігатися

2. Приклад програми

Спочатку відбувається побудова "плану" який оптимізує час обчислень для даної задачі. Потім побудований план передається як параметр функціям, які безпосередньо відповідають за обчислення ШПФ. Приклад одновимірного перетворення комплексного набору даних в комплексний:

```
#include "cufft.h"
#define NX 256
#define BATCH 10
main(){
    cufftHandle plan;
    cufftComplex *data;
    cudaMalloc((void**)&data, sizeof(cufftComplex)*NX*BATCH);

    // створити "план" для БПФ
    cufftPlan1d(&plan, NX, CUFFT_C2C, BATCH);

    // використовувати CUFFT "план" для перетворення.
    cufftExecC2C(plan, data, data, CUFFT_FORWARD);

    // використовувати CUFFT "план" для зворотнього перетворення.
    cufftExecC2C(plan, data, data, CUFFT_INVERSE);

    // видалити CUFFT “план”.
```

```
    cufftDestroy(plan);
    cudaFree(data);
}
```

3. Варіанти завдань

Виконати одновимірне швидке перетворення Фур'є згідно варіанту.

Таблиця 1. Варіанти завдань

№	Розмір даних	Напрям	Тип
1	512	CUFFT_FORWARD	CUFFT_Z2Z
2	1024	CUFFT_FORWARD	CUFFT_Z2Z
3	2048	CUFFT_FORWARD	CUFFT_Z2Z
4	4096	CUFFT_FORWARD	CUFFT_Z2Z
5	8192	CUFFT_FORWARD	CUFFT_Z2Z
6	16384	CUFFT_FORWARD	CUFFT_Z2Z
7	32768	CUFFT_FORWARD	CUFFT_Z2Z
8	512	CUFFT_INVERSE	CUFFT_Z2Z
9	1024	CUFFT_INVERSE	CUFFT_Z2Z
10	2048	CUFFT_INVERSE	CUFFT_Z2Z
11	4096	CUFFT_INVERSE	CUFFT_Z2Z
12	8192	CUFFT_INVERSE	CUFFT_Z2Z
13	16384	CUFFT_INVERSE	CUFFT_Z2Z
14	32768	CUFFT_INVERSE	CUFFT_Z2Z
15	512	CUFFT_FORWARD	CUFFT_C2C
16	1024	CUFFT_FORWARD	CUFFT_C2C
17	2048	CUFFT_FORWARD	CUFFT_C2C
18	4096	CUFFT_FORWARD	CUFFT_C2C
19	8192	CUFFT_FORWARD	CUFFT_C2C
20	16384	CUFFT_FORWARD	CUFFT_C2C
21	32768	CUFFT_FORWARD	CUFFT_C2C

4. Зміст звіту

- Титульний лист;
- Завдання;
- Алгоритм рішення завдання;
- Код програми;
- Екранна форма з результатами роботи програми;
- Висновки.

ДОДАТОК А

НАЛАШТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ ДЛЯ РОБІТ №6-8

Частина 1. Встановлення програмного забезпечення

1.1 Встановлення MS Visual Studio

Для роботи з технологією CUDA буде потрібно MS Visual Studio. Саму останню версію середовища краще не використовувати, оскільки існує ймовірність того, що останній комплект ПЗ CUDA її ще не підтримує.

Зауважимо принагідно, що для навчальних цілей немає різниці у використанні Professional або Express версії Visual Studio. Для роботи з CUDA **необхідно, щоб середовище підтримувала компіляцію програм на мові C/C++**. Тому, якщо Ви встановлюєте Professional версію, проблем, як правило, не буває. При установці Express версії потрібно **звертати увагу на підтримувану мови і тип Вашої операційної системи** (32 або 64 розряду). Express версію можна скачати безкоштовно з сайту Microsoft за посиланням:

[http://www.visualstudio.com/downloads/download-visual-studio-vs]

1.2 Встановлення CUDA Developer Drivers, CUDA Toolkit, GPU Computing SDK

Після встановлення MS Visual Studio необхідно встановити CUDA Developer Drivers, CUDA Toolkit, GPU Computing SDK. Технологія CUDA була розроблена компанією NVIDIA для прискорення обчислень при обробці графіки за рахунок паралельної обробки даних. Прийнято вважати, що зазначена технологія підтримується у всіх відкритих NVIDIA, випущених після 2007 року. Тому якщо у Вас саме така відеокарта - Ви зможете відчути справжню міць паралельних обчислень. Остаточно переконатися в тому, що Ваша відеокарта підтримує технологію CUDA можна на сайті виробника.

Проте якщо у Вас немає відповідного обладнання, вивчити зазначену технологію і навчитися писати програми для CUDA все одно можна. Правда "відчути міць" вже не вийде.

Етапи подальшої установки будуть дещо відрізнятися для конфігурації з встановленою на машині відеокартою від NVIDIA і без неї.

► Для конфігурації ПК з відеокартою від NVIDIA потрібно зайти на сторінку завантаження програмного забезпечення для CUDA,

[https://developer.nvidia.com/cuda-toolkit-archive]

перейти з неї на сторінку з останньою версією (на момент написання це версія 6.5), завантажити і встановити на свій комп'ютер. Починаючи з версії 5.0 всі три пакети, які необхідні для програмування під CUDA, - CUDA Developer Drivers, CUDA Toolkit, GPU Computing SDK - об'єднані в один інсталятор, що, звичайно ж, дуже зручно. При завантаженні звертайте увагу на версію і тип Вашої операційної системи. При установленні не використовуйте довгих шляхів, що містять кириличні букви, а по можливості

короткі шляхи. Це стане в нагоді згодом при інтеграції цих пакетів в Visual Studio.

► Для конфігурації ПК без відеокарти від NVIDIA потрібно зайти на сторінку завантаження програмного забезпечення для CUDA,

[https://developer.nvidia.com/cuda-toolkit-archive]

перейти з неї на сторінку з версією 2.3, завантажити і встановити CUDA Toolkit, GPU Computing SDK. На відміну від першого варіанту конфігурації, пакет CUDA Developer Drivers не потрібен, так як у Вас немає відповідного обладнання. Власне, він навіть не встановиться.

При скачуванні пакетів звертайте також увагу на версію і тип Вашої операційної системи.

При установці CUDA Toolkit, GPU Computing SDK постарайтеся використовувати короткі шляхи без російських букв. Це стане в нагоді згодом при інтеграції цих пакетів в Visual Studio. Найбільш оптимальний варіант - використовувати наступні шляхи установки:

[C:\CUDA\TOOLKIT],

[C:\CUDA\SDK].

Чому необхідно встановлювати ПО CUDA саме версії 2.3? Справа в тому, що саме дана версія - остання в лінійці ПО CUDA, в якій є підтримка режиму емуляції. Отже, якщо включити даний режим в налаштуваннях проекту (про це мова далі), то компілювати й виконувати програмний код CUDA можна і без реального пристрою, що підтримує технологію CUDA.

Не варто також встановлювати CUDA Toolkit, GPU Computing SDK різних версій. Вони можуть виявитися несумісні один з одним.

Після цього в Visual Studio потрібно вказати шляхи до директорій CUDA Toolkit, в яких зберігаються основні заголовні файли, виконувані файли і бібліотеки. Для цього слід відкрити вікно з настройками параметрів ("Сервіс" - "Параметри"), а у вкладці "Проекти та рішення" вибрати пункт "Каталоги VC ++". В якості платформи за замовчуванням слід вибрати "Win32" або "Win64" залежно від розрядності ОС, в полі "Показати каталоги для" вибрати "Виконувані файли", як показано на рис. 1.

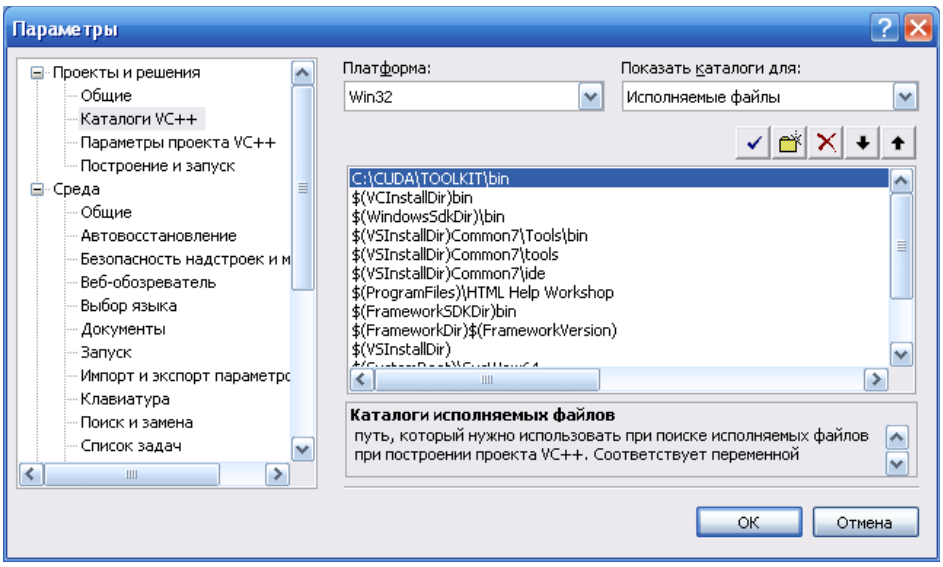


Рис. 1. - Налаштування шляхів до CUDA Toolkit

Далі після натискання кнопки потрібно створити новий рядок і вписати шлях до папки, в якій зберігаються виконувані файли CUDA Toolkit. Як правило, ці файли знаходяться в папці bin, яка, в свою чергу, була створена при установці CUDA Toolkit. Таким чином, повний шлях, який слід вказати в налаштуваннях Visual Studio, виглядає так:

[C:\CUDA\TOOLKIT\bin].

Далі слід перевірити існування зазначеного шляху, натиснувши кнопку . Якщо введений шлях некоректний, з'явиться вікно з повідомленням про помилку; якщо ж всі шляхи із зазначеного списку виконуваних файлів існують, нічого не відбудеться. Не варто нехтувати цією перевіркою: можна бути повністю впевненим, що шлях зазначений коректно (хоча насправді це не так) і витратити багато часу в пошуках помилок компіляції або відсутніх бібліотек в майбутньому.

Аналогічно необхідно вказати шляху до заголовки та бібліотекам CUDA Toolkit. При цьому в полі "Показати каталоги для" слід вибирати "Включаються файли" і "Файли бібліотек". Всі заголовні файли бібліотеки CUDA Toolkit знаходяться в папці include, всі бібліотеки - в папці lib відповідно.

Після вказівки шляхів до заголовки та бібліотекам не забувайте перевіряти їх коректність, щоразу натискаючи на кнопку .

Частина 2. Створення та налагодження першого проекту для CUDA

Матеріал, описуваний у цій частині, показує, як створити проект, налаштувати інтеграцію встановленого раніше програмного забезпечення, скомпілювати і запустити перший додаток для роботи з CUDA. Зверніть увагу - при створенні нового проекту настройка повинна виконуватися щоразу заново, тому, скоріше за все, Вам доведеться повертатися до даної частини повторно.

Для початку створимо новий консольний додаток Win32 C++. У налаштуваннях створення **позначимо опцію "Порожній проект"**. При створенні проекту **не використовуйте папку для проектів за замовчуванням, а намагайтеся використовувати короткі шляхи без кириличних букв**. Найбільш оптимальний варіант - створити папку:

[C:\CUDA\PROJECTS]

і створювати всі проекти в ній. Принагідно зазначимо, що дане правило створення проектів актуально для версії CUDA ToolKit версії 2.3. Можливо в наступних версіях (а зараз вже вийшла 6.5) даний недолік був виправлений.

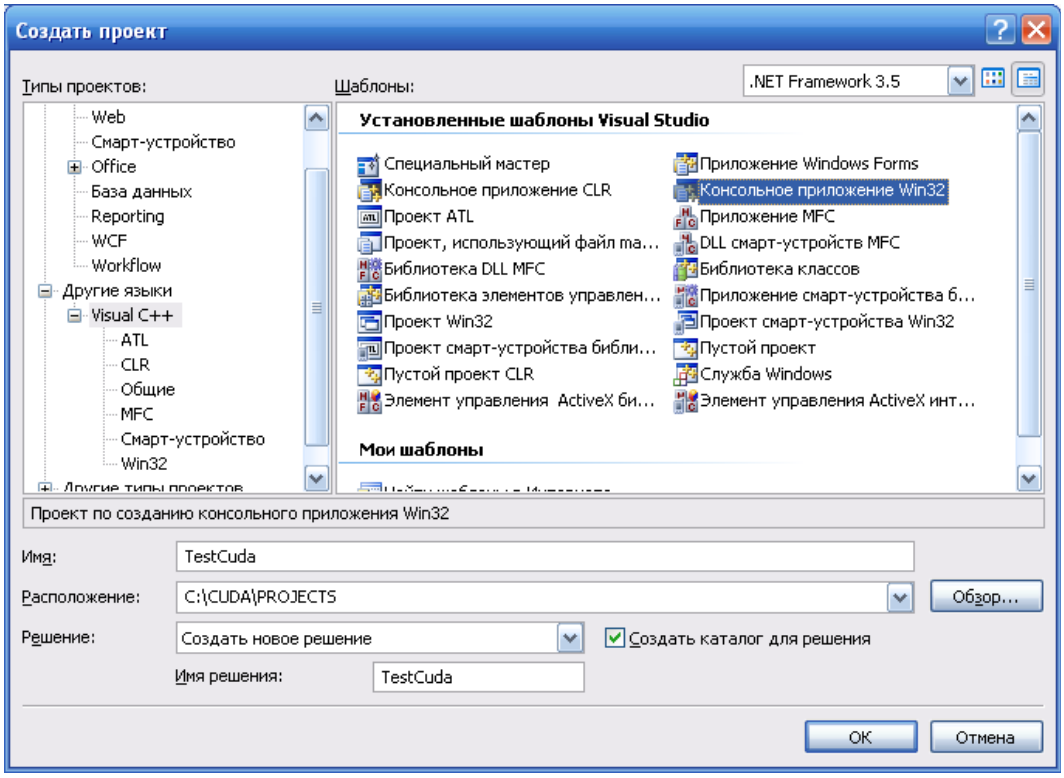


Рис. 2. - Створення нового проекту CUDA (1 крок)

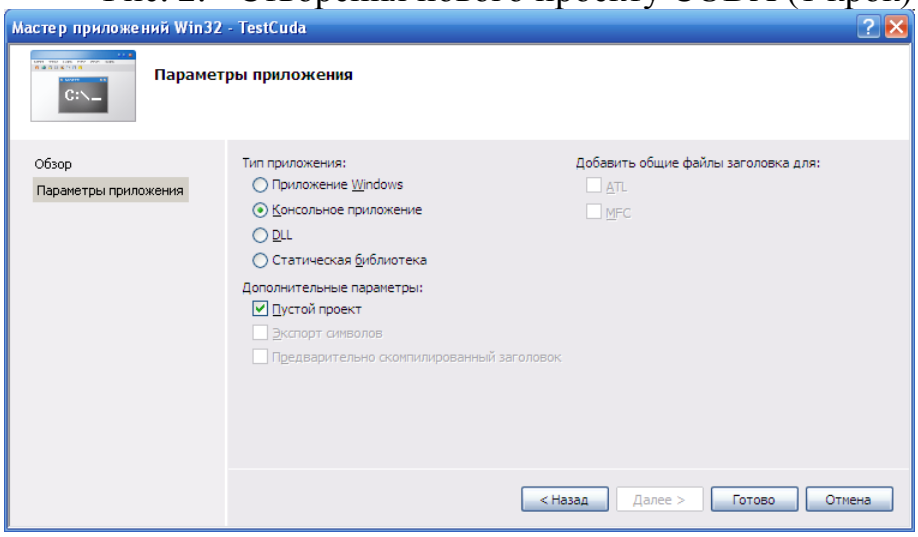


Рис. 3. - Створення нового проекту CUDA (2 крок)

Після створення нового проекту Вам слід перевірити, чи додалися правила складання в Visual Studio. В останніх версіях CUDA Toolkit це виконується автоматично. Правила складання можуть не додаватися в старих версіях CUDA Toolkit. Крім того, правила складання не будуть додані, якщо Ви, наприклад, встановите спочатку CUDA Toolkit, а тільки потім Visual Studio - в цьому випадку їх просто ще нікуди додавати.

Перевірити чи додалися правила складання можна через контекстне меню проекту.

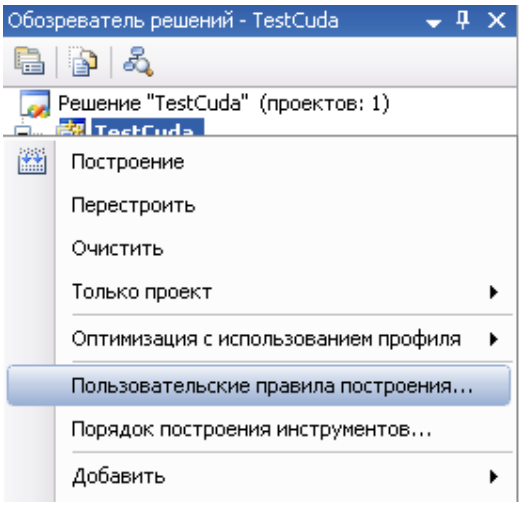


Рис. 4. - Виклик діалогу Користувальницькі правила побудови

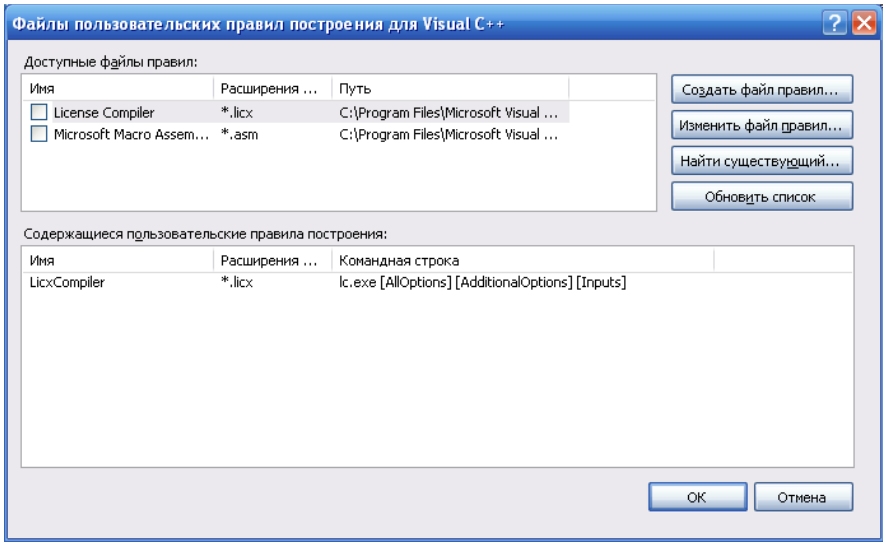


Рис. 5. - Діалог "Користувальницькі правила побудови"

У діалозі "Користувальницькі правила побудови" в назвах правил шукайте рядок "CUDA". Таких правил може бути декілька, а їх повні назви можуть відрізнятися залежно від версії CUDA Toolkit("CUDA C/C ++", "CUDA Driver API", "CUDA Runtime API" і т.д.).

Якщо подібних правил в зазначеному діалозі немає (як вийшло, наприклад, на рис. 5), Вам потрібно додати їх самостійно. Для цього в папці з встановленим пакетом CUDA Toolkit пошукайте файл з назвою [Cuda.Rules]. Якщо такого файлу немає, спробуйте пошукати його в папці з встановленим пакетом GPU Computing SDK (в старих версіях файл Cuda.Rules може

перебувати в ньому). Таких файлів Ви можете знайти декілька, в різних піддиректоріях. Додайте всі знайдені правила вручну за допомогою кнопки "Знайти існуючий". В результаті діалогове вікно прийме вигляд, показаний на рис. 6.

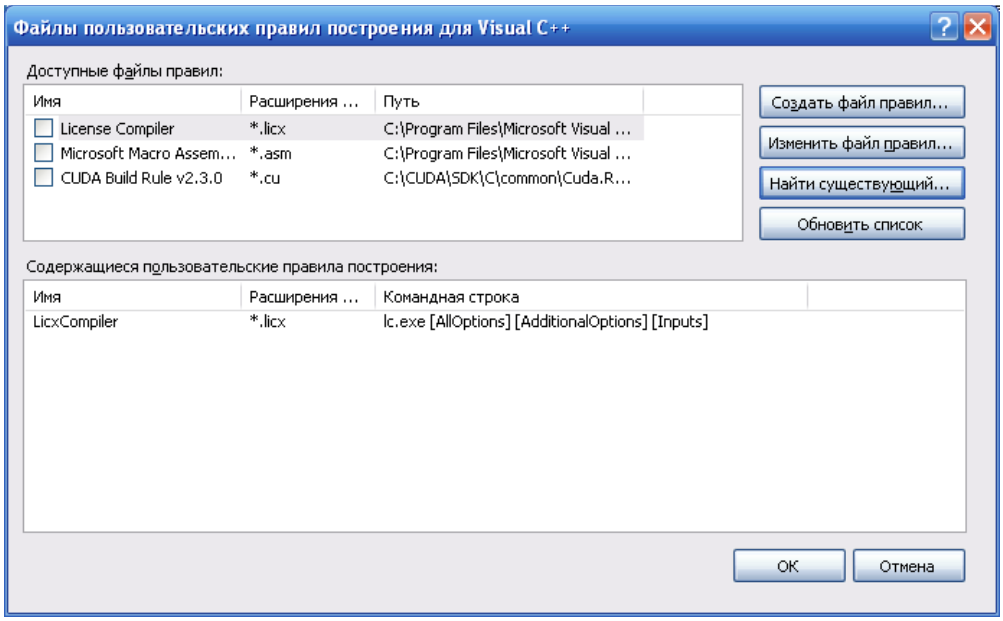


Рис. 6. - Діалог "Користувальницькі правила побудови" з доданим правилом збірки для CUDA-програм

Для того щоб задіяти правила складання в Вашому проекті, необхідно відзначити прапорець зліва від назви правила і натиснути кнопку "ОК". Якщо ж у Вас є кілька правил (в старших версіях CUDA Toolkit, наприклад, є поділ на CUDA Runtime API і CUDA Driver API), слід вибирати CUDA Runtime API. Саме CUDA Runtime API надає інтерфейс для більш простих програм CUDA, в той час як CUDA Driver API більш складний в освоєнні, але разом з тим дає доступ до низкорівневого програмування для CUDA. Для початківців він не потрібен.

На наступному етапі потрібно створити новий файл з вихідним кодом в проекті. Для цього слід скористатися пунктом "Створити елемент" з контекстного меню проекту (див. Рис. 7). Далі потрібно вибрати файл типу C ++. Файл може бути будь-яким, але **розширення файлу обов'язково слід задавати вручну**, як показано на рис. 8. Традиційно файли вихідного коду CUDA мають розширення *.cu. Якщо Ви не задасте розширення самостійно, вказаний файл буде звичайним *.cpp файлом, та додані раніше правила для нього стануть непридатні. З малюнка 6 можна зрозуміти, що саме по розширенню файлу Visual Studio визначає, які правила для нього потрібно задіяти і як його компілювати.

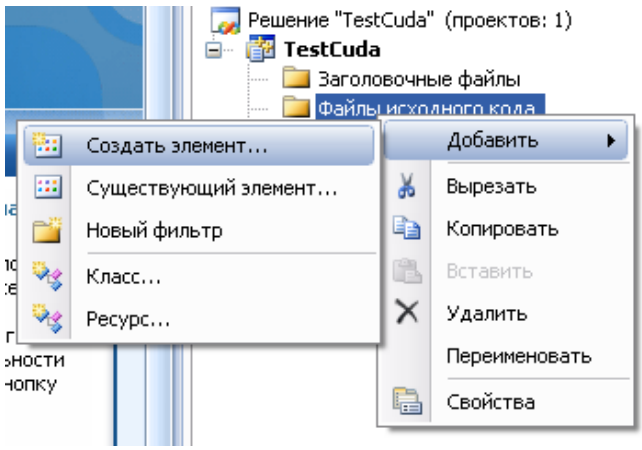


Рис. 7. - Створення нового файлу з вихідним кодом (крок 1)

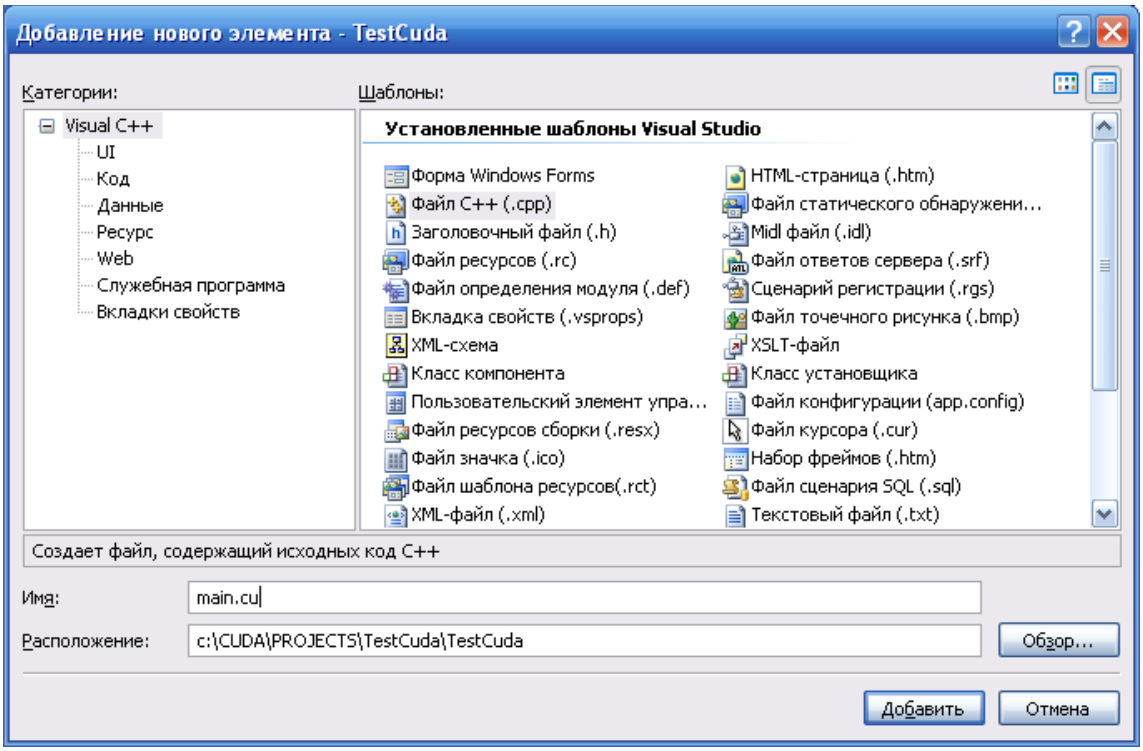


Рис. 8. - Створення нового файлу з вихідним кодом (крок 2)

Після створення файлу слід переконатися в тому, що до нього застосувались правила CUDA. Для цього потрібно відкрити властивості файлу (не проекту!) І переконатися в тому, що в поле "Інструмент" вкладки "Загальні" задіюється саме підключення правила складання CUDA (див. Рис. 9 - рис. 10). Якщо з яких-небудь причин в полі інструмент виявиться інший інструмент - можна перепризначити інструмент вручну, вибравши його зі списку.

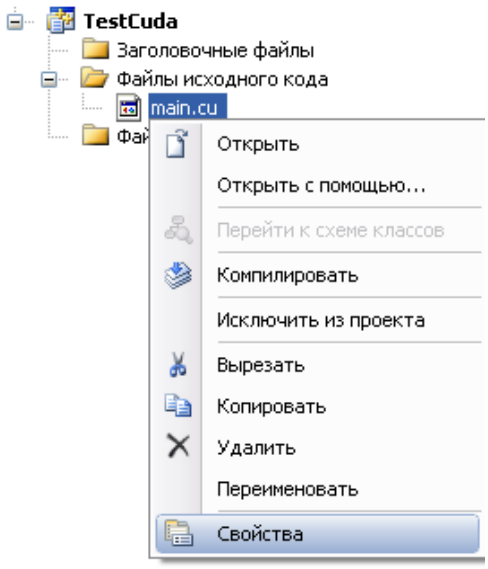


Рис. 9. - Перевірка відповідності вихідного файлу правилам CUDA (крок 1)

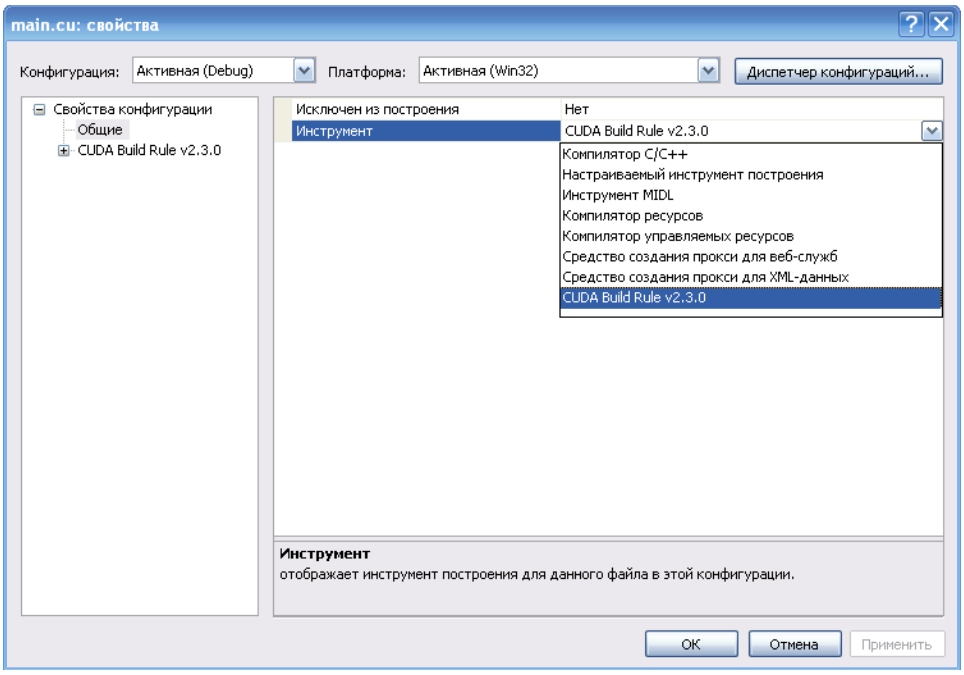


Рис. 10. - Перевірка відповідності вихідного файлу правилам CUDA (крок 1)

Далі слід налаштувати шляхи для пошуку середовищем Visual Studio компілятора CUDA програм, бібліотек CUDA, заголовків файлів та інших виконуваних файлів (які входять до складу CUDA ToolKit). Знадобиться виконати кілька налаштувань у властивостях проекту:

- По-перше, потрібно вказати місце розташування бібліотек CUDA у вкладці "Компоновщик - Загальні - Додаткові каталоги бібліотек", як показано на рис. 11. Тут слід вказати шлях до папки, в якій зберігаються бібліотеки CUDA (файли * .lib). Ця папка входить до складу CUDA ToolKit. Якщо таких папок у Вашій версії CUDA ToolKit кілька, то, потрібно вибрати ту з них, яка відповідає типу Вашої операційної системи та обраної версії CUDA API. Наприклад, якщо файли * .lib знаходяться в папці "runtime / lib /

win32 / *. Lib" і у Вас 32 розрядна версія Windows, то вибирати потрібно саме цю папку.

- По-друге, слід вказати бібліотеку "cudart.lib" на вкладці "Компоновщик - Загальні - Введення". Тут можна обмежитися одним ім'ям файлу, без вказівки шляху (див. Рис. 12).

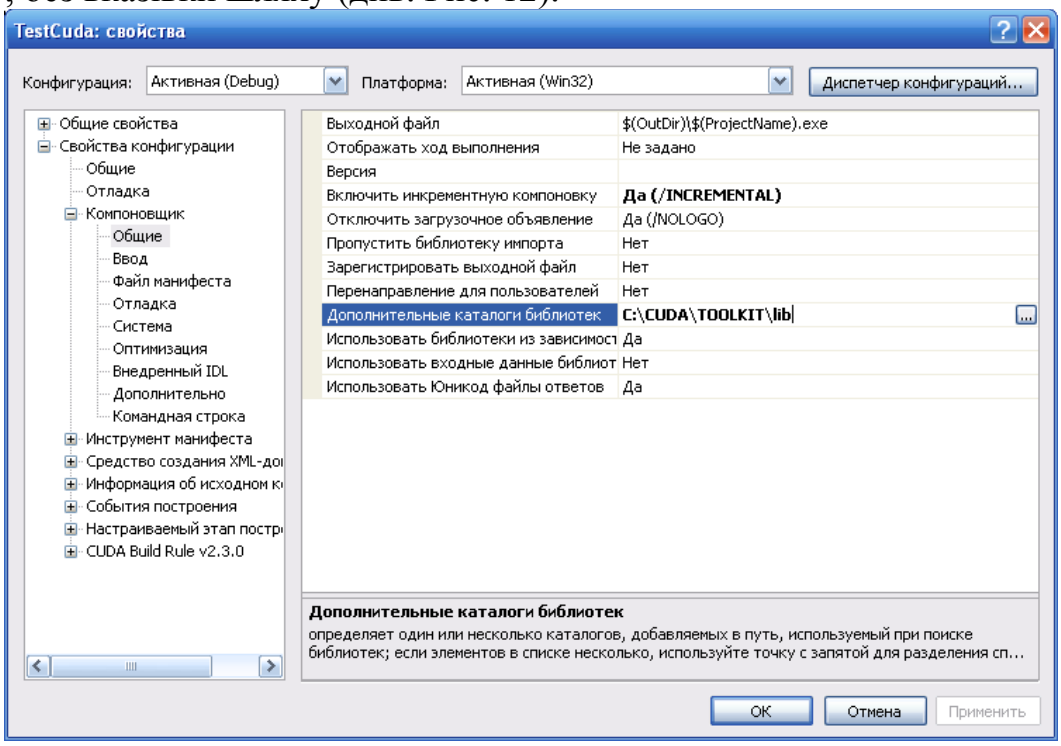


Рис. 11. - Підключення бібліотек CUDA

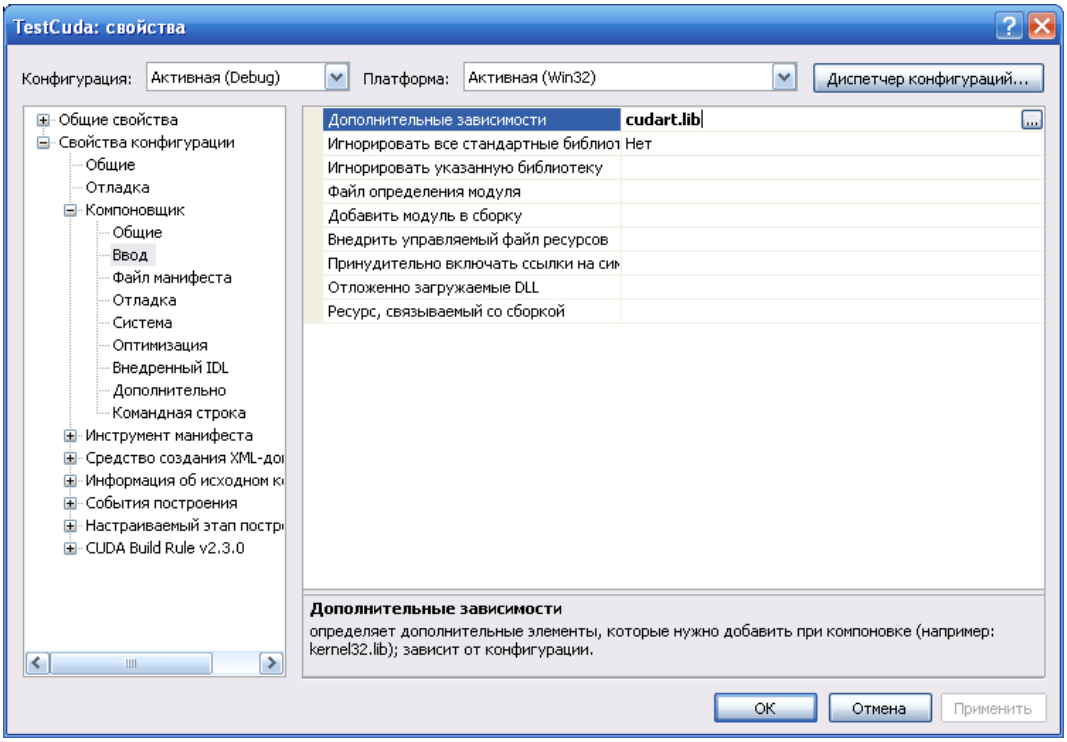


Рис. 12. - Підключення cudart.lib

Нарешті, **для студентів, у яких немає реальної відеокарти від NVidia, що підтримує CUDA, необхідно включити режим емуляції CUDA**

пристрою. Зверніть увагу на наступний цікавий факт: якщо режим емуляції не включений, то програма компілюється і навіть працює, окрім ділянок коду, які повинні виконуватися на пристрої CUDA. Тому будьте уважні і **завжди включайте режим емуляції**, інакше результат виконання Вашої програми буде не таким, як Ви очікували. Для того щоб включити режим емуляції у властивостях проекту зайдіть у вкладку "CUDA Build Rule v2.3.0 - General - Emulation Mode" і виберіть "Так" навпроти відповідного властивості (див. Рис. 13).

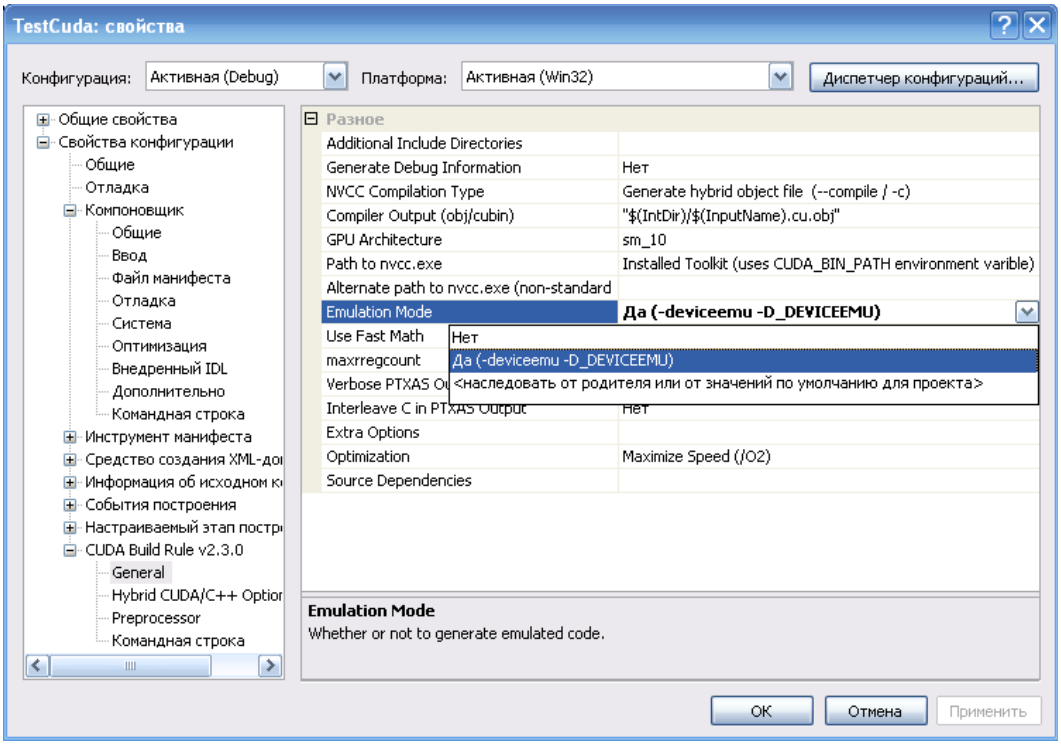


Рис. 13. - Включення режиму емуляції (для CUDA ToolKit v2.3.0)

Тепер варто перевірити, як компілюється найпростіший додаток CUDA. Відкрийте створений раніше файл вихідного коду і введіть у нього наступний фрагмент:

```
#include <stdio.h>

__global__ void add(int a, int b, int *c)
{
   *c = a + b;
}

int main(void)
{
   int c = 0; int *dev_c;

   cudaMalloc ((void**)&dev_c, sizeof(int));
   add<<<1,1>>>(2, -1, dev_c);
   cudaMemcpy (&c, dev_c, sizeof(int), cudaMemcpyDeviceToHost);

   printf("%d", c);
   system("pause");
   return 0;
```

```
}
```

Неважко здогадатися (незважаючи на місцями незнайомий синтаксис), що робить цей фрагмент коду: складає 2 числа і виводить результат на екран. Але відбувається це вже на CUDA пристрої. Тому якщо в результаті Ви побачите 1, значить, Ви все зробили правильно. Якщо в результаті буде 0, то, скоріше за все, Ви забули включити режим емуляції. Можуть також виникнути і помилки компіляції. У цьому випадку спробуйте скомпілювати такий код:

```
int main(void)
{
   return 0;
}
```

Якщо і в цьому випадку будуть виникати помилки компіляції, то можна перевірити наступне:

- Якщо виникають помилки виду "newline в константі", то ця помилка, скоріше за все, пов'язана з тим, що в шляху до проекту або в шляху до бібліотек є кириличні літери;

- Якщо виникають помилки виду "нерозв'язане посилання" — також ще раз перевірте правильність шляхів до всіх бібліотекам.

- Нарешті, при компіляції проектів Ваш антивірус може "лаятися". Просто додайте компілятор nvcc.exe в список дозволених додатків. В ньому вірус точно відсутній, за умови, звісно, що Ви завантажили CUDA Toolkit з офіційного сайту NVidia.